

Аудит работ партнёра — Кострюков И.А. (Задания 1–4)

Проект партнёра: Steam Market Price Predictor — веб-сервис прогнозирования цен на активы торговой площадки Steam

Аудитор: Кулакова Софья

Методология оценки: Задание 5, скоринговая модель Задания 2, макс. балл за каждое задание -100.

Задание 1 — Конкурентный анализ, SWOT, бенчмаркинг, SLA

Итоговая оценка: 82 из 100 — Отлично

Конкуренты выбраны адресно, SWOT охватывает пять конкурентов, бенчмаркинг содержит колонку «Наше решение», SLA структурированы по трём категориям и содержат обоснования. Снижение происходит преимущественно из-за формата бенчмаркинга (символы вместо чисел) и нескольких методологических ошибок в SLA.

Детализация по критериям

Блок/Критерий	Макс	Мой балл	Обоснование
Выбор конкурентов (×2)	8	8	5 прямых конкурентов: CSFloat, SteamAnalyst, Pricempire, CSGOStocks, Skinsbook. Покрывают все ключевые ниши (маркетплейс, агрегатор, ТА-инструмент, заявленный прогнозировщик). Обоснование по каждому конкуренту есть. Полный балл.
Обоснование выбора (×2)	8	6	Каждый конкурент описан с конкретными данными (9.3 млн посещений у CSFloat, 30+ маркетплейсов у SteamAnalyst). Но нет единого критерия отбора «почему именно эти пять, а не Skinport или Buff163» — это важные игроки, исключённые без объяснения.
Полнота SWOT (×3)	12	10	SWOT выполнен для всех 5 конкурентов. По CSFloat и SteamAnalyst — отличный уровень детализации. По Pricempire и Skinsbook — немного поверхностнее, по 3–4 пункта на квадрант вместо 4–5.

Методологическая корректность SWOT (×2)	8	7	S/W/O/T разделены корректно. Замечание: у Skinsbook в секции «Угрозы» присутствует «Зависимость от бесплатной модели» — это скорее внутренняя слабость (W), а не внешняя угроза (T). Методологическая ошибка.
Глубина анализа (×2)	8	7	Для CSFloat и SteamAnalyst — хорошая конкретика. CSGOStocks описан хорошо через призму OHLC и мониторинга пула дропов. У Skinsbook «Непрозрачная методология» упомянута, но не проанализированы последствия для конкурентной позиции.
Бенчмаркинг (×2)	8	5	Бенчмаркинг содержит 14 критериев и колонку «Наше решение» — это хорошо. Однако используется формат ✓/~/– без числовых шкал. Невозможно понять, насколько именно «Наше решение» лучше по ключевым параметрам. Например, по «Точности прогноза» — у нашего ✓, у Skinsbook ~. Но какова фактическая разница в MAPE?
Обоснование критериев бенчмаркинга (×1)	4	3	Критерии логичны (прогнозирование, бэкестинг, API, Docker). Нет объяснения, почему «Docker-контейнеризация» включена как конкурентный критерий — это внутренняя характеристика, не видимая пользователям.
Выводы по конкурентному анализу (×1)	4	2	Итоговых выводов нет. Нет раздела «В чём «Наш проект» имеет максимальное конкурентное преимущество и над кем конкретно».
Полнота SLA (×3)	12	12	5+5+5 метрик по трём категориям (IT, Ops, BD). Полный комплект.
Формализация метрик SLA (×2)	8	7	Все метрики числовые: $\geq 99.5\%$, ≤ 5 сек, 0 критических ошибок. Исключение: BD-метрика «Воспроизводимость экспериментов: CSV с метаданными» — это формат, а не SLA.
IT-SLA (×2)	8	7	Uptime 99.5%, RTO ≤ 30 мин — обоснованы. Отсутствует RPO (Recovery Point Objective), что является стандартной парой к RTO.

Операционный SLA (×2)	8	7	Хорошие метрики. «Воспроизводимость экспериментов: CSV с метаданными» — это не SLA, а стандарт хранения данных.
BD SLA (×2)	8	6	«Обновление модельного стека ≥ 1 новой модели / 6 мес» — это roadmap, а не SLA. SLA описывает уровень сервиса сейчас, а не план развития. «Время вывода MVP новой функции ≤ 4 недели» — аналогично, это KPI разработки, а не SLA.
Обоснование SLA (×2)	8	7	Обоснование есть у всех метрик, часть — с хорошей глубиной (объяснение, почему потеря данных Steam API критична для ML-моделей).
Реалистичность SLA (×1)	4	3	В целом реалистично. Uptime 99.5% при Docker Compose на одном VPS — сложно достижимо без hot standby. Это должно быть отражено как ограничение или требование к инфраструктуре.
Связь с ВКР (×1)	4	4	Связь прослеживается явно: SLA напрямую связаны с качеством ML-моделей и Data Pipeline.

Замечания по блокам

Замечание 1. Стр. 13, бенчмаркинг, формат ✓/~/–:

Нет числовых шкал. «Прогнозирование: ✓» у нашего решения и «~» у Skinsbook — в чём разница? MAPE 8% vs 20%? Автоматически vs эвристически? Без цифр бенчмаркинг не позволяет принять решение.

→ -3 балла по критерию «Бенчмаркинг».

Замечание 2. Стр. 17–18, BD SLA — roadmap-метрики:

«Обновление модельного стека ≥ 1 новой модели / 6 мес» — это не SLA. SLA фиксирует текущий уровень услуги, а не план развития. Аналогично: «Время вывода MVP ≤ 4 недели». Это тайм-трекинг разработки.

→ -2 балла по критерию «BD SLA».

Замечание 3. Стр. 15, IT-SLA — отсутствует RPO:

Есть RTO ≤ 30 минут. Но нет RPO (до какой точки можно восстановить данные). При потере

Steam API данных за 4 часа — каков допустимый временной лаг восстановления? Без RPO SLA неполон.

→ -1 балл по «IT-SLA».

Рекомендации

- **P0.** Заменить ✓/~/– в бенчмаркинге на числовые шкалы (1–5 или 1–10) по каждому критерию. Ожидаемый прирост: +3 балла.
- **P0.** Убрать roadmap-метрики из BD SLA, заменить на реальные уровни сервиса (например: «Время ответа на запрос API ≤ 200 мс при нагрузке ≤ 100 RPS»). Ожидаемый прирост: +2 балла.
- **P1.** Добавить RPO к IT-SLA. Ожидаемый прирост: +1 балл.
- **P1.** Добавить раздел итоговых выводов после SWOT. Ожидаемый прирост: +2 балла.
- **P2.** Объяснить отсутствие Skinport и Buff163 в списке конкурентов.

Задание 2 — ITIL и регламенты

Итоговая оценка: 71 из 100 — Хорошо

Работа демонстрирует глубокое понимание принципа «меньше — лучше» применительно к масштабу проекта: выбор трёх ITIL-процессов вместо попытки охватить весь v4-каталог — методологически зрелое решение. Структура документа чёткая, регламенты читаются, логика передана. Тем не менее ряд существенных методологических пробелов не позволяет поставить оценку выше.

Детализация по критериям

Блок 1: ITIL-процессы (28/40 взвешенных баллов)

Критерий	Вес	Макс	Мой балл	Обоснование
Выбор релевантных ITIL-процессов	×2	8	7	Выбраны три процесса: Incident Management, Change Management, Service Level Management. Все три релевантны проекту и закрывают ключевые риски: потерю данных,

				регрессию при изменениях, контроль SLA. Небольшое снижение за то, что не рассмотрен Problem Management — при системе из 17 моделей с зависимостью от Steam API повторяющиеся сбои API являются ожидаемым паттерном, и отсутствие корневого анализа — пробел.
Обоснование выбора процессов	×2	8	7	Обоснование по каждому процессу конкретное и привязано к коду: упомянуты steam_client.py, <u>prediction.py</u> , MinMaxScaler, 17 моделей. Это выше среднего по курсу. Минус балл: не показана критичность именно <i>трёх</i> (а не двух или пяти) — почему Problem Management или Knowledge Management исключены, не сказано явно.
Описание целей процессов	×1	4	3	Цели сформулированы чётко, например: «Обеспечить максимально быстрое восстановление работоспособности сервиса при сбоях». Немного снижаю за то, что цель SLM звучит описательно («Определить, зафиксировать и контролировать»), но не указывает бизнес-результат — для кого и зачем (пользователи прогнозного сервиса, потеря ценовых рядов).
Определение границ процессов	×1	4	4	Лучший раздел работы. Для каждого процесса явно прописано «в границы входит» и «в процесс не входит». Это полностью корректно методологически. Пример: «В Incident Management не входят: глубокий анализ корневой причины повторяющихся сбоев» — строгое разграничение с Problem Management.
Описание ролей процессов	×1	4	3	Четыре роли (Владелец, Разработчик, Оператор, Аналитик данных) консистентны по всем трём процессам — хорошо. Снижение: роль «Оператор системы» в контексте одиночного студенческого проекта де-факто совпадает с ролью «Разработчика» — не показано, как разрешается это дублирование ролей на практике.
Описание этапов процессов	×2	8	7	Все три процесса имеют пронумерованные этапы (8, 8, 7 шагов соответственно). Последовательность логична. Минус балл: в Change Management отсутствует явный rollback-этап — что делать, если внедрённое изменение сломало модели?

				Для системы с 17 ML-моделями это критически важный пропуск.
Связь процессов между собой	×2	8	7	Раздел 2 «Взаимосвязь выбранных процессов» присутствует и описывает замкнутый контур: SLM → Incident → Change → SLM. Диаграмма или схема потоков отсутствует — только текст. Снижение за отсутствие формализованной карты триггеров (например: «Инцидент класса P1 → автоматически инициирует Change Request»).

Итог Блока 1: 38/44 → нормированный вес: ~26/28

Блок 2: Матрица RACI (24/40 взвешенных баллов)

Критерий	Вес	Макс	Мой балл	Обоснование
Наличие матрицы RACI	×2	8	7	Матрица присутствует, содержит 12 действий и 4 роли. Охватывает все три процесса. Минус балл: строки не сгруппированы по процессам — читается как единая таблица, из которой неясно, к какому процессу относится «Оценка влияния на качество».
Логика распределения ролей	×2	8	6	В целом логично: Оператор — R/A на мониторинге, Разработчик — R/A на изменении кода. Но есть проблема: «Устранение инцидента» — Оператор A/R, Разработчик C . Если инцидент — ошибка в коде (что типично для этого проекта), Оператор не может быть Accountable за то, что не в его компетенции. А должен быть у Владельца, R у Разработчика.
Отсутствие конфликтов ответственности	×1	4	2	Критическое замечание. В матрице присутствуют комбинации A/R у одной роли : «Мониторинг выполнения парсинга» (Оператор A/R), «Регистрация инцидента» (Оператор A/R), «Исправление ошибки в коде» (Разработчик A/R), «Тестирование изменения» (Разработчик A/R). По методологии ITIL/RACI, R и A могут быть у одного человека только если роль достаточно изолирована. Наличие A/R одновременно в

				ключевых операционных действиях означает отсутствие независимого надзора — это антипаттерн, особенно для контроля качества.
Связь ролей с процессами	×1	4	3	Роли повторяются идентично во всех трёх процессах — это последовательно. Но нет явной строчки «процесс / владелец процесса» — неясно, кто несёт ITIL-style Process Owner ответственность за каждый из трёх процессов.
Соответствие масштабу проекта	×1	4	3	4 роли для одиночного / студенческого проекта — разумно. Однако в Задании 4 появляется роль «ML-инженер», которой нет в RACI Задания 2. Это нарушение связности между заданиями.

Итог Блока 2: 21/28 → нормированный вес: ~21/28

Блок 3: Регламенты (нет отдельного весового блока в скоринге, входит в общую оценку)

Регламенты для всех трёх процессов присутствуют. Каждый содержит: Цель, Область применения, Ценность, Входы, Выходы, Последовательность действий. Это соответствует требованиям задания.

Достоинства регламентов:

- Входы и выходы конкретны (например, входы для Incident: «сообщение об ошибке», «лог выполнения парсера», «результат мониторинга»)
- Ценность для организации прописана во всех трёх случаях
- Регламент Change Management содержит явную привязку к тестовой среде

Замечания по регламентам:

- Ни один регламент не содержит **временных SLO** (например: «Инцидент класса Critical должен быть зарегистрирован за ≤ 5 минут, эскалирован за ≤ 15 минут»). Без таймингов регламент — это описание, а не исполнимый документ

- Отсутствует ссылка на **шаблоны/тикеты** — как именно регистрируется инцидент? Jira? Telegram? Markdown-файл?
- В регламенте Change Management нет критериев отклонения изменения (go/no-go критерии)

Итог по регламентам: ~16/20 (оценка встроена в общий пул)

Итоговый расчёт

По скоринговой модели из задания:

Критерий	Вес	Балл	Итог
Выбор ITIL-процессов	×2	3 (из 4)	6
Обоснование выбора	×2	3	6
Описание целей	×1	3	3
Определение границ	×1	4	4
Описание ролей	×1	3	3
Описание этапов	×2	3	6
Связь процессов	×2	3	6
Наличие матрицы RACI	×2	3	6
Логика распределения ролей	×2	3	6
Конфликты ответственности	×1	2	2
Связь ролей с процессами	×1	3	3
Соответствие масштабу	×1	3	3
Итого	×19 (сумма весов)	—	54 / 76 max

Нормировка к 100 баллам: $54 / 76 \times 100 \approx 71$ баллов → **Хорошо**

Сильные стороны работы

1. **Принцип минимальной достаточности процессов** (стр. 1, введение): Явное объяснение, почему выбраны именно 3 процесса, а не весь каталог ITIL.
2. **Чёткое определение границ процессов** (стр. 2, 4, 6): Каждый процесс содержит секцию «в процесс не входит». Это предотвращает scope creep и показывает зрелость архитектурного мышления.
3. **Замкнутый управленческий контур** (стр. 9, раздел 2): Явная формулировка: «SLM → Incident → Change → SLM».
4. **Привязка к коду проекта** (стр. 3–5): Обоснование выбора процессов ссылается на конкретные модули: `steam_client.py`, `prediction.py`, бэктестинг.
5. **Консистентность ролей**: Четыре роли стабильны по всем трём процессам и по всему документу.

Замечания по блокам

Замечание 1. Стр. 9, RACI, строки «Мониторинг парсинга», «Регистрация инцидента», «Устранение инцидента»:

Оператор системы стоит одновременно R и A. Это конфликт: тот, кто исполняет, не может одновременно принимать результат. Особенно критично для «Устранения инцидента», где причина — почти всегда ошибка в коде (т.е. зона ответственности Разработчика).

Предлагаю: A → Владелец сервиса, R → Оператор (оперативные) / Разработчик (технические).

→ Критерий «Конфликты ответственности»: -2 балла из 4.

Замечание 2. Стр. 7–8, все регламенты — отсутствие временных SLO:

Регламенты описывают последовательность шагов, но не содержат ни одного временного ограничения. Для Incident Management это делает документ декларативным, а не исполнимым. Без таймингов («зарегистрировать за ≤ 5 мин», «разрешить P1 за ≤ 30 мин») нельзя проверить соблюдение регламента.

→ Снижает оценку по критерию «Описание этапов процессов»: -1 балл.

Замечание 3. Стр. 5, Change Management — отсутствие rollback-этапа:

В 8 шагах Change Management нет ни rollback, ни rollback-критериев. Для системы с 17 ML-моделями, где изменение параметров MinMaxScaler или feature engineering может одновременно «сломать» несколько моделей, отсутствие плана отката — критический пробел.

→ Снижает «Описание этапов процессов».

Замечание 4. Задание 2 vs Задание 4, несоответствие ролей:

В Задании 4 (стр. ТСО, раздел «Роли») появляется «ML-инженер» как самостоятельная роль. В RACI Задания 2 этой роли нет — функции ML растворены в «Аналитике данных» и «Разработчике». При переходе к production-системе это приведёт к неопределённости ответственности за retraining и model deprecation.

Рекомендации по улучшению

- **P0 — Критичное.** Устранить конфликты A/R в RACI: перенести Accountable на Владельца сервиса во всех операционных действиях, оставив R исполнителям. Ожидаемый прирост: +2 балла.
- **P0 — Критичное.** Добавить тайминги в регламент Incident Management: время регистрации, время эскалации, время разрешения по классам (P1/P2). Ожидаемый прирост: +1 балл.
- **P1 — Желательное.** Добавить rollback-шаг и go/no-go критерии в Change Management. Ожидаемый прирост: +1 балл.
- **P1 — Желательное.** Ввести роль «ML-инженер» в RACI или явно указать, что она входит в «Аналитик данных». Это синхронизирует Задание 2 с Заданием 4. Ожидаемый прирост: +1 балл.
- **P1 — Желательное.** Добавить схему/диаграмму взаимосвязи процессов (хотя бы ASCII или таблицу триггеров). Ожидаемый прирост: +1 балл.
- **P2 — Nice to have.** Добавить Process Owner к каждому процессу как отдельную позицию в RACI (ITIL-стандарт). Ожидаемый прирост: +0,5 балла.

Потенциал: при выполнении P0 + P1 оценка вырастет с 71 до ~79 баллов (верхняя граница категории «Хорошо» / начало «Отлично»).

Задание 3 — Метрики, алерты, дашборд

Итоговая оценка: 84 из 100 — Отлично

Семь метрик выбраны с явной привязкой к бизнес-ценности, методика измерения прописана для каждой, пороги трёхуровневые (норма/warning/critical), алерты маршрутизированы по ролям. Дашборд структурирован по аудиториям.

Детализация по критериям

Критерий	Макс	Мой балл	Обоснование
Выбор метрик (привязка к бизнесу)	16	14	7 метрик: Uptime, Predict latency, Steam API success rate, MAPE, Error Rate, Incidence count, Data completeness. Все привязаны к бизнесу — особенно MAPE («единственная метрика, измеряющая качество основного продукта»). Снижение: нет метрики активных пользователей / конверсии — это чисто product health метрика, которая показывает бизнес-здоровье, а не только техническое.
Методика измерения (источник данных, формула, воспроизводимость)	16	15	Для каждой метрики: источник (Prometheus, FastAPI logs, /api/backtest), формула расчёта, периодичность. MAPE формула приведена явно. Снижение: для «Количество инцидентов» методика — «ручная регистрация или автоматическая», это оставляет неопределённость.
Реалистичность порогов	12	10	Пороги обоснованы. Однако: Uptime warning < 99.5% и critical < 99.0% — разрыв всего 0.5%. На практике при Docker Compose на одном VPS переход из warning в critical будет мгновенным при любом сбое. Нет «буфера реагирования». Для predict latency: норма ≤10 сек — слишком мягко. Если LSTM требует 25+ сек, это системная проблема, а не warning.

Маршрутизация алертов (роль, канал, runbook)	12	10	Warning → Email → Оператор; Critical → Telegram + Email → Оператор + Разработчик + Владелец. Чётко. Снижение: нет runbook-ссылки — при Critical alert «Analyse logs» это инструкция, а не runbook. Не указано время эскалации: если оператор не ответил за X минут, кто следующий?
Макет дашборда (структура, аудитория, иерархия)	12	10	4 секции с явной привязкой к аудиториям: операционная команда (секции 1–2), аналитик (секция 3), менеджмент (секция 4). Иерархия данных продумана — от «светофора» (здоровье одним взглядом) до журнала инцидентов. Снижение: дашборд описан текстом, нет даже ASCII-схемы или wireframe. Инструмент реализации не указан (Grafana? DataDog?).
Связь метрик с ITIL-процессами	12	10	Явно упомянуто, что алерты связаны с RACI из Задания 2 и инициируют Change Management. Это системная связь. Снижение: нет формального mapping-таблицы: «Метрика X → триггерит Process Y → Accountable Role Z».

Итог: 69/80 нормированных → ~86/100

С учётом отсутствия wireframe дашборда и ряда неточностей: **84/100**.

Сильные стороны

1. **Метрика MAPE как product-метрика** (стр. 5): Редкий случай, когда студент включает метрику качества ML-модели в операционный мониторинг. Это показывает понимание специфики продукта.
2. **Разделение Steam API errors и собственных ошибок** (стр. 4): «502 = Steam, 500 = наш код, 422 = невалидные данные» — правильная диагностическая декомпозиция.
3. **Привязка алертов к конкретным действиям** (стр. 8–9): Каждый critical алерт содержит конкретное действие (перезапуск контейнера, проверка cookie), а не просто «разобраться».

4. **Аудиторно-ориентированный дашборд** (стр. 10–11): Явное разделение на операционную команду и менеджмент.

Замечания

Замечание 1. Стр. 9, алерты — отсутствие runbook-ссылок:

«Analyse logs: 502 = Steam, 500 = our code» — это диагностика, но не runbook. Runbook должен содержать пошаговые действия, ссылки на документы, команды (например: docker-compose restart app). Без runbook реагирование зависит от памяти дежурного.

Замечание 2. Стр. 7, метрика «Количество инцидентов» — неопределённость регистрации:

«Ручная регистрация или автоматическая» — это не методика. Если инциденты регистрируются вручную, метрика будет занижена. Нужно: «Инцидент автоматически создаётся при срабатывании critical alert; ручная регистрация — только для проактивно выявленных проблем».

Задание 4 — TCO, ROI, финансовая модель

Итоговая оценка: 80 из 100 — Хорошо

Задание 4 — наиболее зрелая аналитическая работа. Финансовая модель содержит CAPEX, OPEX, три источника дохода, ROI, срок окупаемости, три альтернативы и явные допущения. Это выходит за рамки минимальных требований. Снижение вызвано системными ограничениями модели (нет NPV/дисконтирования, нет детализации источников цен) и оптимистичностью ряда допущений.

Детализация по критериям

Критерий	Макс	Мой балл	Обоснование

Полнота ТСО (CAPEX + OPEX, источники цен)	20	16	CAPEX детализирован (6 статей), OPEX — по трём годам с логикой роста. Хорошо. Снижение: отсутствуют источники цен. «Ставка разработчика 150 тыс/мес» — откуда? hh.ru ? Glassdoor? «VPS 15 тыс/мес» — это Timeweb? Yandex Cloud? Без источников цифры нефальсифицируемы.
Формулы ROI и срока окупаемости	16	14	Формула ROI корректна: $(\text{Выгоды} - \text{Затраты}) / \text{Затраты} \times 100\%$. Срок окупаемости определён как момент перехода накопленного потока в ноль. Снижение: ROI рассчитан без дисконтирования. На горизонте 3 лет и при ставке привлечения капитала 20%+ (стартапы в РФ) NPV существенно ниже. Допущение «инфляция не учитывается» явно указано, но не обосновано достаточно.
Реалистичность допущений	16	12	Допущения прописаны в разделе 6.1. MAU 3k/10k/25k — агрессивный, но обоснованный план при активном маркетинге в нишевых сообществах. Конверсия 2–6% — реалистична по SaaS-бенчмаркам. Критичное допущение: «Steam API остаётся бесплатным» — это не просто допущение, это SPOF всей бизнес-модели. Valve вводила ограничения API в 2023–2025 гг. Сценарий монетизации Steam API не разработан.
Качество анализа альтернатив	16	14	Три альтернативы: не запускать, Build vs Buy (SaaS), продать прототип. Все три реалистичны и содержат плюсы/минусы. Снижение: «Build vs Buy» — ТСО посчитан только для Enterprise SaaS (DataRobot, H2O.ai). Не рассмотрен вариант open source (AutoML, MLflow + собственный хостинг), который значительно дешевле.
Структура и ясность документа	12	10	Документ хорошо структурирован: бизнес-контекст → IT-инициатива → ТСО → выгоды → ROI → допущения → альтернативы → рекомендация. Снижение: раздел «Косвенные выгоды» содержит неточности — «снижение churn на 15–20% по бенчмаркам SaaS» без указания конкретного источника.

Связь с предыдущими заданиями	10	8	Явно упомянуто, что ITIL-процессы (Задание 2) и мониторинг (Задание 3) уже спроектированы и снижают операционные риски. ТСО включает «Обучение команды процессам ITIL» как статью CAPEX.
Бизнес-контекст и обоснование инвестиций	10	8	6 бизнес-целей с метриками и управленческими решениями. ROI 68.1% — конкретная цифра с выводом. Снижение: рекомендация к инвестированию «подтверждена» без указания альтернативной нормы доходности (hurdle rate). Для чего 68.1% — это хорошо? Лучше банковского депозита (15%)? Лучше ОФЗ (12%)?

Сильные стороны

1. **Детализация CAPEX по статьям** (стр. ТСО раздел 3.1): 6 статей с обоснованием каждой (FTE × месяцы × ставка). Это не «потратим примерно миллион», а модель.
2. **Три источника дохода** (стр. 4.1–4.3): Premium, B2B API, реклама. Диверсификация.
3. **Явные ограничения модели** (стр. 6.3): Раздел честно признаёт: «не учитываем NPV», «не учитываем налоги», «ARPU — среднее по году».
4. **Анализ альтернатив** (стр. 7): Три сравнимых варианта с итоговой таблицей ROI.

Замечания

Замечание 1. Стр. ТСО/3.1 — источники цен на ресурсы:

150 тыс/мес — ставка разработчика. VPS 15 тыс/мес. Нет ни одной ссылки. В бизнес-документе источники цен обязательны: «по данным [hh.ru](#) median, май 2025» или «Yandex Cloud calculator».

Замечание 2. Стр. 5.2 — отсутствие NPV:

ROI 68.1% звучит убедительно, но без дисконтирования это номинальная доходность. При стоимости капитала для early-stage 25–30% (типично для РФ), NPV проекта существенно ниже. Даже одна строка NPV-расчёта показала бы финансовую зрелость.

Замечание 3. Стр. 6.1 — допущение «Steam API бесплатен»:

Это не просто допущение — это основной бизнес-риск. Альтернативный сценарий (Steam API стал платным / закрытым) должен быть проработан как отдельный стресс-тест ТСО.

Оценка устойчивости к «чёрным лебедям» и качества Recovery-протоколов

Числовая оценка устойчивости: 4/10

Система спроектирована как MVP с одиночными точками отказа по всем критическим компонентам. Это соответствует масштабу проекта, но при переходе в production категорически недостаточно.

Матрица Black Swans

Сценарий	Вероятность	Влияние	Готовность системы
Steam API вводит платный доступ / закрывается	Средняя	Критическое	✗ Не готова. Scraped-данные есть, но нет формального плана перехода и нет альтернативного источника данных
Массовый DDoS на VPS	Низкая	Высокое	✗ Нет CDN-защиты, rate limiting не описан для внешних атак
Потеря cookie steamLoginSecure (используется для Steam API)	Высокая	Критическое	⚠ Частично. Есть алерт «проверить cookie», но нет автоматической ротации
10-кратный всплеск нагрузки (вирусный пост о сервисе)	Средняя	Высокое	✗ Docker Compose на одном VPS не масштабируется горизонтально
Valve изменяет формат ответа Steam API	Высокая	Критическое	✗ steam_client.py привязан к конкретному формату; нет version detection
Потеря ключевого разработчика	Средняя	Высокое	⚠ Частично. CLAUDE.md задокументирован, но нет onboarding checklist и knowledge base
Обвал рынка скинов CS2 на 50%+	Низкая	Высокое	⚠ Диверсификация на Dota 2/TF2 предусмотрена в плане, но не реализована
Санкционные ограничения на облачные сервисы	Средняя	Среднее	⚠ Частично — выбор российского VPS обоснован

Blast Radius Containment

Критическое наблюдение: Система построена на архитектуре «всё в одном» — Docker Compose, один VPS, один Steam API endpoint. Нет разделения на:

- Data Collection Tier (парсинг Steam API)
- Processing Tier (ML-модели)
- Serving Tier (FastAPI endpoint)

Сбой Steam API **полностью блокирует** /api/history и /api/predict. Нет graceful degradation: «если Steam API недоступен, вернуть кэшированные данные». Bulkhead pattern отсутствует.

Качество Recovery-протоколов

Аспект	Оценка	Комментарий
RTO	✓ Определён	≤30 минут в SLA (Задание 1)
RPO	✗ Не определён	Не указано, какой потери данных допустима
DR Plan	✗ Отсутствует	Нет формального Disaster Recovery Plan
Runbook for top-5 failures	△ Частично	Алерты содержат действия, но нет структурированного runbook
Роль принимающего DR-решение	✗ Не указана	В RACI нет строки «Решение об активации DR»
DR Drills	✗ Не предусмотрены	Нет chaos engineering или плановых учений
Blameless post-mortems	✗ Отсутствуют	Нет механизма накопления знаний по инцидентам

Топ-3 критических пробела в recovery-дизайне

1. **Отсутствие RPO и DR Plan.** Есть RTO, но нет RPO. Что происходит при потере 6 часов ценовых данных? 24 часов? Это влияет на качество всех 17 моделей и должно быть явно задокументировано.
2. **SPOF: Steam API cookie.** Единственный механизм аутентификации к Steam API (steamLoginSecure cookie) — это одиночная точка отказа с высокой вероятностью: cookie истекают, блокируются, требуют обновления. Автоматической ротации нет.
3. **SPOF: один VPS без горизонтального масштабирования.** При вирусном росте (что реалистично для нишевого сервиса после упоминания у стримера) система упадёт без возможности быстрого масштабирования.

Рекомендации по устойчивости

- **P0.** Определить RPO и задокументировать допустимую потерю данных. Это займёт 2 часа, но закроет критический пробел в SLA.
- **P0.** Реализовать кэширование последних N дней данных как fallback при недоступности Steam API. scraped_data/ уже существует в проекте — использовать как резервный источник.
- **P1.** Описать механизм автоматической ротации Steam API cookie или переключения на альтернативный API-ключ.
- **P1.** Создать runbook для top-3 сценариев (Steam API down, VPS down, MAPE exceeded) — хотя бы в формате markdown-файла в репозитории.
- **P2.** Запланировать первый DR drill через 3 месяца после production-запуска.

Итоговые рекомендации по всем четырём работам

Приоритет	Рекомендация	Задание	Ожидаемый эффект
P0	Устранить конфликты A/R в RACI; добавить тайминги в регламент Incident Management	Задание 2	+3 балла Задание 2; снизит риск «аудит без исполнения»
P0	Определить RPO; реализовать fallback на кэшированные данные при недоступности Steam API	Задание 3 + Recovery	Снижает критический SPOF бизнес-модели
P0	Заменить ✓/~/– в бенчмаркинге на числовые шкалы	Задание 1	+3 балла Задание 1; делает анализ принятием решения
P1	Добавить rollback-этап в Change Management	Задание 2	+1 балл; критично для системы с 17 ML-моделями
P1	Добавить NPV-расчёт к ROI; указать источники цен для TCO	Задание 4	+3 балла Задание 4; делает финансовую модель проверяемой

Карта рисков работ (heatmap вероятность × влияние)

	Низкое влияние	Среднее влияние	Высокое влияние
Высокая вероятность	—	Конфликты A/R в RACI (32)	Steam API SPOF без RPO; Cookie rotation (33/Recovery)
Средняя вероятность	—	Формат ✓/~/– в бенчмаркинге (31)	Нет rollback в Change Mgmt (32); Нет NPV в ROI (34); Нет DR Plan

Низкая вероятность	Нет итоговых выводов по SWOT (31)	Нет runbook-ссылок в алертах (33)	—
-------------------------------	--------------------------------------	--------------------------------------	---

Сводная оценочная таблица

Задание	Оценка	Категория	Потенциал при исправлении P0+P1
Задание 1 — Конкурентный анализ, SWOT, SLA	82/100	Отлично	~89/100
Задание 2 — ITIL и регламенты	71/100	Хорошо	~79/100
Задание 3 — Метрики, алерты, дашборд	84/100	Отлично	~90/100
Задание 4 — TCO, ROI, финансовая модель	80/100	Хорошо	~87/100
Средняя оценка	79.25/100	Хорошо	~86/100